PLANT DISEASE DETECTION USING CONVOLUTIONAL NEURAL NETWORKS

Deep Learning Laboratory Project Report

Student Name	P. Laxmi Kanth Reddy
Programme	B.Tech — Computer Science and Engineering
Course	Deep Learning Laboratory
Institution	ICFAI Foundation for Higher Education
Academic Year	2026–2027
Faculty / Guide	_____

Before submission: run the code, insert the measured results and generated plots, and fill in the selected dataset classes. No experimental scores are pre-filled.

Abstract

Plant diseases can reduce crop quality and yield. This project develops an image-classification prototype that uses a Convolutional Neural Network (CNN) to classify leaf images into selected healthy and diseased plant categories. Images are resized and normalized, and light data augmentation is applied during training. The model is trained using TensorFlow/Keras and evaluated with accuracy, per-class precision, recall, F1-score, and a confusion matrix. A Streamlit interface is included for demonstration. This is an educational prototype; results on a controlled dataset do not establish reliability in real agricultural settings.

1. Introduction

Deep learning enables computer systems to learn visual patterns from examples. CNNs are commonly used for image classification because convolutional filters learn local features such as edges, textures, spots, and shapes. In plant disease classification, these features may help distinguish healthy leaves from leaves showing disease symptoms. This project demonstrates a workflow from dataset preparation to model evaluation and inference.

2. Problem Statement and Objectives

Problem statement: Manual identification of plant diseases from leaf images may be difficult for non-specialists. This project investigates whether a CNN can classify leaf images into predefined dataset categories.
Objectives:
• Prepare a labeled dataset and create training, validation, and test subsets.
• Implement a CNN using TensorFlow/Keras.
• Train with regularization and monitor validation performance.
• Evaluate with multiple classification metrics.
• Provide an optional image-upload demo and document limitations.

3. Literature Review

CNN-based image classifiers are a common approach to plant disease recognition. Mohanty, Hughes, and Salathé (2016) explored deep learning for image-based plant disease detection using labeled leaf images. Their work demonstrated the potential of convolutional networks while also highlighting the gap between controlled image datasets and real-world conditions. This project uses a smaller custom CNN so students can understand the architecture and training process. It does not claim to reproduce published benchmark results.

4. Dataset and Preprocessing

The proposed source is the PlantVillage dataset. Download it from the Kaggle or GitHub source linked in README.md and place images in class-named subfolders under `data/raw/`. For a short lab timeline, use three to five classes with adequate images per class. Images are resized to 128 × 128 RGB pixels and scaled to [0, 1]. Training augmentation applies horizontal flips, small rotations, and zoom. The supplied loader creates a reproducible random split. Inspect per-class counts and duplicates; for formal evaluation, prefer a fixed, stratified split manifest.
Selected classes: [Fill in]
Total images: [Fill in]
Split counts: [Fill in after preparation].

5. Methodology and Model Design

The model accepts a 128 × 128 × 3 image. Three convolutional blocks use 32, 64, and 128 filters with ReLU activation. Max pooling reduces spatial dimensions, and batch normalization helps stabilize optimization. Global average pooling aggregates feature maps, followed by a 128-unit dense layer, dropout, and a softmax output with one unit per class. The model uses Adam, sparse categorical cross-entropy, early stopping, and learning-rate reduction.

Component	Configuration / purpose
Input	128 × 128 RGB image, normalized to [0, 1]
Augmentation	Random horizontal flip, rotation, zoom
Convolution blocks	32 → 64 → 128 filters; 3 × 3 kernels; ReLU
Downsampling	MaxPooling2D + BatchNormalization

Component	Configuration / purpose
Classifier	GlobalAveragePooling → Dense(128) → Dropout
Output	Softmax, one output per class
Training	Adam; sparse categorical cross-entropy; batch size 32; up to 15 epochs

Implementation files: src/model.py, src/data_utils.py, train.py, evaluate.py, predict.py, and app.py.

6. Implementation Procedure

1. Install dependencies from requirements.txt in a virtual environment.
2. Download the dataset and organize it into one folder per class.
3. Run `python train.py` to train the model.
4. Review learning curves and test metrics in outputs/.
5. Run `python evaluate.py` to generate the classification report and confusion matrix.
6. Run `python predict.py path/to/image.jpg` to classify an image.
7. Optionally launch the demo using `streamlit run app.py`.

7. Results and Discussion

The model has not been trained as part of this source package because the image dataset must be downloaded separately. Therefore, no measured accuracy or F1-score is claimed here. Run the scripts and insert the actual values and generated plots before submitting.
Test accuracy: [Enter outputs/metrics.json value]
Macro F1-score: [Enter outputs/classification_report.json value]
Training time: [Record observed time]
Observations: [Discuss strongest and weakest classes and the confusion matrix]. Compare training and validation curves for overfitting. Accuracy can be misleading for imbalanced classes, so include per-class precision, recall, and F1-score.

8. Conclusion

This project provides an end-to-end educational implementation of CNN-based plant leaf image classification. It covers preprocessing, model construction, training, evaluation, and inference. Its effectiveness must be judged from held-out test results produced after dataset preparation and training.

9. Future Scope

Possible extensions include transfer learning with MobileNetV2, a larger and more diverse dataset, class balancing, Grad-CAM explanations, mobile deployment, and external testing on field photographs. These extensions should not be presented as completed work unless implemented.

10. References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection,” Frontiers in Plant Science, vol. 7, article 1419, 2016. DOI: 10.3389/fpls.2016.01419.
[2] PlantVillage Dataset, Kaggle: https://www.kaggle.com/datasets/emmarex/plantdisease (accessed on: [insert date]).
[3] TensorFlow, “Load and preprocess images,” https://www.tensorflow.org/tutorials/load_data/images.
[4] scikit-learn, “Classification report,” https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html.

11. Appendix: Commands and Viva Preparation

Commands:

```
pip install -r requirements.txt
python train.py
python evaluate.py
streamlit run app.py
```

Viva topics: CNN vs. fully connected networks; convolution and pooling; ReLU and softmax; cross-entropy; overfitting and dropout; train/validation/test split; accuracy vs. precision/recall/F1; class imbalance; and generalization to field

images.
Limitations: The prototype predicts only among its trained labels. It cannot reliably detect unseen classes and is not professional agricultural advice.